

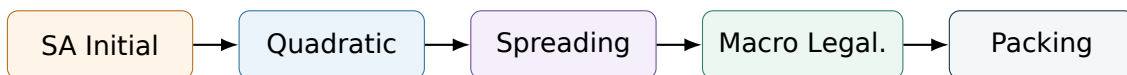
AMF-Placer

第六天学习笔记

全论文复盘、算法耦合关系与源码阅读准备

今天不再逐段推进论文，而是建立 “AMF-Placer 的完整心智模型”。

目标是把前五天学到的 SA Initial Placement、Quadratic Placement、Cell Spreading、Progressive Macro Legalization、Packing 与实验结果串成一条完整的数据流，并明确哪些机制是本文提出、哪些来自已有工作。最后给出一个只依赖论文术语的源码检索路线，方便下一阶段进入实现。



本日覆盖范围

- 重新定义论文真正解决的问题：FPGA analytical mixed-size placement
- 用 Fig. 3 重新串联 7 个阶段以及 lower-bound / upper-bound 迭代
- 复盘 5 个核心技术 Tech1-Tech5 的作用与相互依赖
- 复盘 macro 对 QP、spreading、legalization 和 packing 各自造成的困难
- 用 Table II / III 与 Fig. 6 重新理解实验结论和论文边界
- 给出 “paper-to-code” 阅读清单：基于论文关键词搜索实现，不虚构仓库目录结构

论文：T. Liang et al., “AMF-Placer: High-Performance Analytical Mixed-size Placer for FPGA,” ICCAD 2021.

目录

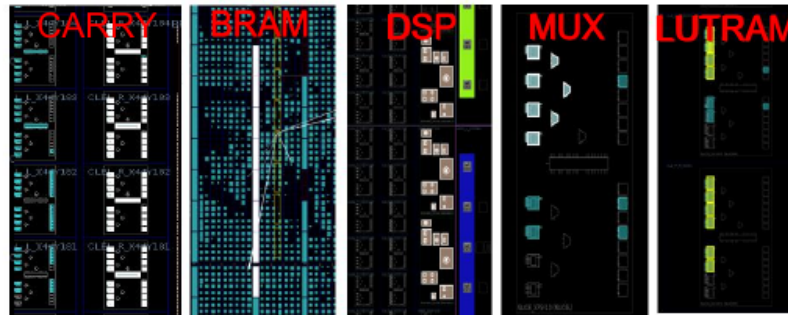
1 第一件事：重新回答“这篇论文究竟解决什么？”	2
2 完整 7 阶段流程：不要再把算法看成孤立模块	3
3 Lower-bound 与 Upper-bound：整个闭环的骨架	4
4 Tech1 复盘：为什么先做 SA-based Initial Placement?	5
5 Tech2 复盘：Quadratic Placement + Pseudo Net 到底解决什么?	6
6 Tech3 + Tech4 复盘：Cell Spreading 为什么是 mixed-size 的难点?	7
7 Tech5 复盘：Progressive Macro Legalization 的两层逻辑	8
8 Exact Legalization 的 DP：把二维问题压成列内一维问题	10
9 Packing 与 Resource Feedback：为什么它们不是收尾杂项?	11
10 把 Tech1-Tech5 放到同一张因果图中	12
11 实验复盘：Table II / III 应该怎样读?	13
12 Fig. 6 与论文边界：AMF-Placer 并不等于 Vivado placer	14
13 哪些部分是本文重点创新，哪些是继承已有工作?	15
14 Paper-to-Code：下一阶段怎样开始读源码?	16
15 源码阅读时最值得打断点/打印日志的量	17
16 如果要复现实验，最小闭环应该是什么?	18
17 Day 6 自测：你现在是否真正掌握了整篇论文?	19
18 六天学习总览与下一步	20

1 第一件事：重新回答“这篇论文究竟解决什么？”

AMF-Placer 的问题不是一般意义上的“FPGA placement”，而是：

Analytical FPGA placement + Mixed-size macros

传统 analytical FPGA placer 更容易处理“一个 instance 对应一个 site/BEL”的标准单元情形。AMF-Placer 强调现实设计中存在大量由多个 standard cells 组成、具有相对位置和资源约束的 macros，例如 CARRY、MUX、LUTRAM、DSP cascade 和 BRAM cascade。



ig. 2. Example of various types of macros with shape constraints: the

图 1: 论文 Fig. 2: 不同类型 macro 的示例。图像直接截取自原论文。

为什么“mixed-size”会改变问题性质？

一个 macro 不只是“面积更大的 cell”。它往往同时具有：多 cell 形状约束、资源类型约束、跨多个 site 的占用、高外部连接度，以及与其它 standard cells 细粒度共享 CLB 的可能性。因此 QP、spreading、legalization、packing 都必须改变。

2 完整 7 阶段流程：不要再把算法看成孤立模块

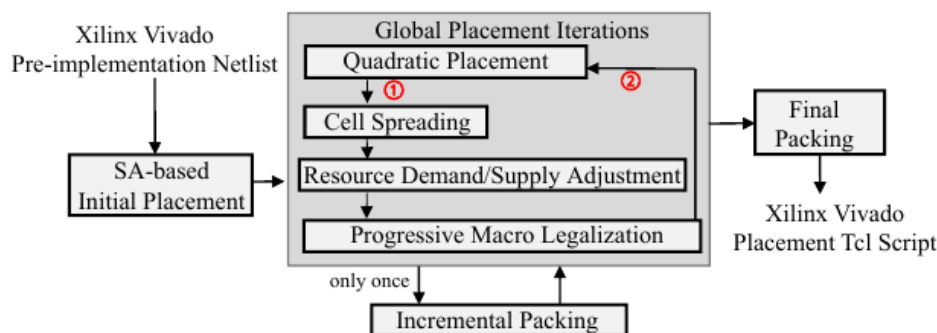


Fig. 3. The Outline of AMF-Placer's Workflow with 7 Process Phases: Label 1 indicates lower-bound placement and label 2 indicates upper-bound placer.

4) *Resource Demand/Supply Adjustment*: Based on the packing feasibility and routing congestion level, the area

图 2: 论文 Fig. 3: AMF-Placer 的完整 workflow, 直接裁取自原论文。

论文的 7 个阶段可以重新整理成三层:

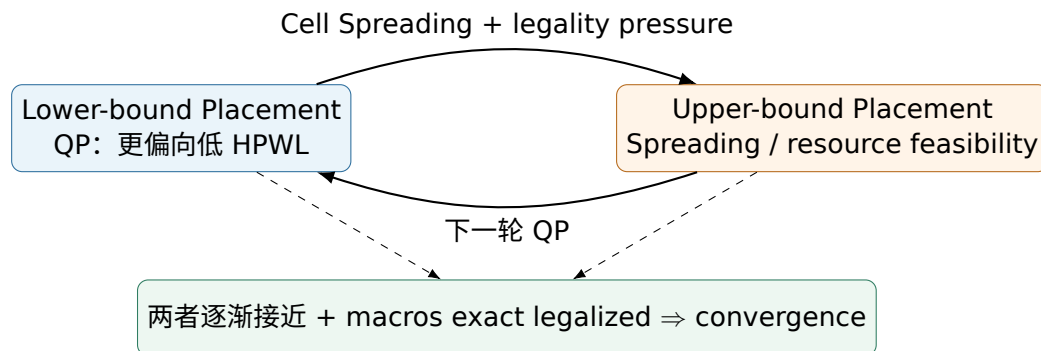
1. 全局搜索层: SA-based Initial Placement;
2. **Global placement** 迭代层: Quadratic Placement、Cell Spreading、Resource Demand/Supply Adjustment、Progressive Macro Legalization, 以及只执行一次的 Incremental Packing;
3. 最终离散实现层: Final Packing。

最容易忽略的点

Fig. 3 不是“执行 7 步然后结束”的线性流水线。Quadratic Placement 与 Cell Spreading 之间形成 lower-bound / upper-bound 的迭代, macro legalization、resource feedback 和 incremental packing 共同影响下一轮位置。因此真正的算法是一个逐渐收敛的闭环。

3 Lower-bound 与 Upper-bound: 整个闭环的骨架

论文把 quadratic placement 的结果称为 lower-bound placement，因为它更接近连续优化意义下的低 HPWL；而经过 spreading 等可实现性修正后的 placement 被称为 upper-bound placement。



可以把收敛直觉写成：

$$HPWL_{\text{lower}} \approx HPWL_{\text{upper}}$$

同时 macro 已经进入 exact legalization 状态。

理解 AMF-Placer 的统一视角

QP 负责“优化”，Spreading/Legalization/Packing 负责“把优化结果拉回 FPGA 可实现空间”。论文的多数新技术，本质上都在减少这两者之间的冲突和震荡。

4 Tech1 复盘：为什么先做 SA-based Initial Placement?

AMF-Placer 先用 PaToH 递归二分 netlist, 将 instances 聚成 clusters, 再把 FPGA 划成粗粒度 bins, 在 cluster 层做 simulated annealing。论文的动机是：异构 FPGA 的资源分布不均, analytical model 容易出现局部能量极小值；随机因素可以提供大范围移动能力。

初始 SA 的目标函数由线长与 bin overflow penalty 组成：

$$F_{\text{cost}} = \sum_{e \in E} W_e + \sum_{BC_{ij} \in B_C} Of(BC_{ij}).$$



论文支持的结论

消融实验中，去掉 Tech1 对具有清晰层次结构的 OptimSoC、OpenPiton 影响很大。论文因此把高质量 initial placement 视为 analytical placement 的必要过程之一。

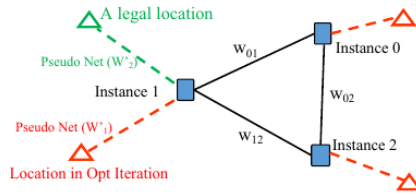
5 Tech2 复盘: Quadratic Placement + Pseudo Net 到底解决什么?

HPWL 本身含有 max 和绝对值, 不适合直接作为简单二次目标。因此 AMF-Placer 采用 Bound2Bound weighted quadratic model:

$$\widetilde{W}_e = \sum_{i,j \in e} [w_{x,ij}^{B2B} (x_i - x_j)^2 + w_{y,ij}^{B2B} (y_i - y_j)^2].$$

真正的 mixed-size 问题在于: FPGA 合法位置是离散、异构且受 macro shape 约束的。论文通过 virtual anchors 与 pseudo nets, 把“合法性压力”软性加入 QP。

tance of the corresponding instance in last optimization



4. An Example of Pseudo Nets and Anchors: the dash lines indicate pseudo nets and the triangles represent anchors.

图 3: 论文 Fig. 4: 实例、anchor 与 pseudo nets, 直接裁取自原论文。

以前的方法:

$$w_{e_p, v_i} = \frac{\alpha}{\text{movement}(v_i)}.$$

AMF-Placer 修改为:

$$w_{e_p, v_i} = \frac{\alpha}{\text{movement}(v_i)} \times \text{pinNum}(v_i).$$

Tech2 的本质

高 pin-count macro 一旦大幅移动, 会同时改变大量 nets, 使当前 quadratic wirelength approximation 更容易失真。乘以 pinNum 后, 连接度高的 macro 对 anchor 的“重力”更强, 移动更保守, 从而稳定 QP 迭代。

6 Tech3 + Tech4 复盘: Cell Spreading 为什么是 mixed-size 的难点?

普通 spreading 的目标是把资源 demand 超过 supply 的 overflowed region 中的 cells 向外扩散。但 macro 会跨多个 bins, 直接整体移动可能把 overflow 从一个区域搬到另一个区域, 形成 deadlock。



Fig. 5. An Example of Cell Spreading Deadlock of Macro

图 4: 论文 Fig. 5: Macro 导致 cell spreading deadlock, 直接裁取自原论文。

AMF-Placer 的关键处理包括:

- 对反复 overflow 的 bins 注入 resource supply fluctuation;
- spreading window 不按固定模板扩张, 而是优先向相邻 utilization 最低的方向扩张;
- 构造互不重叠的 spreading windows, 使多个窗口可以并行;
- 两阶段 macro spreading: 先临时释放 macro 内 standard cells 的 shape constraint, 再用这些 cells 的平均位置更新 macro;
- 用 forgetting rate 限制后期 spreading 更新幅度。

Forgetting rate:

$$\gamma = 1 - 0.95 \frac{HPWL_{lower}}{HPWL_{upper}}.$$

位置更新:

$$p_i^{U'} = \gamma p_i' + (1 - \gamma) p_i^U.$$

为什么 Tech3 与 Tech4 是一组?

Tech3 尽量让 spreading window “只扩大到需要的地方”; Tech4 则控制 “即使决定 spread, 也不要一步走得太猛”。一个控制空间范围, 一个控制更新幅度, 两者共同减少 HPWL 破坏。

7 Tech5 复盘: Progressive Macro Legalization 的两层逻辑

Algorithm 2: Progressive Macro Legalization

Input: upper-bound placement $\mathbf{P}^U = \{(x_i^U, y_i^U)\}$, candidate site number for each cell/macro N_{cand} , initial displacement threshold θ_{disp_I} , threshold increment $\Delta\theta_{disp}$ and displacement threshold to enable exact legalization θ_{exact}

Output: psuedo nets connected to legalization anchors for macros $E_p^{Legal} = \{e_p^{Legal}\}$

```

1 unmappedCells = findUnmappedCellsInMacros();
2 cell2SiteRoughMap =  $\emptyset$ ;
3  $\theta_{disp} = \theta_{disp\_I}$ ;
4 while  $size(unmappedCells) > 0$  do
5   cell2CandidateSiteMap =
     findCandidateSites(unmappedCells,  $\theta_{disp}$ ,  $N_{cand}$ );
6   BPGraphs = createIndependentBipartite-
     Graphs(cell2CandidateSiteMap);
7   parallel foreach  $G$  in  $BPGraphs$  do
8      $\lfloor$  minCostBipartiteMatching( $G$ );
9   updateRoughLegalization(cell2SiteRoughMap);
10  unmappedCells = findUnmappedCellsInMacros();
11   $\theta_{disp} = \theta_{disp} + \Delta\theta_{disp}$ ;
12 avgDisp = checkAvgDisp(cell2SiteRoughMap);
13 if  $avgDisp > \theta_{exact}$  then
14    $\lfloor E_p^{Legal} = linkToAvgLoc(cellSiteRoughMap)$ 
15 else
16   macro2ColumnMap = spreadMacrosToCols( $\mathbf{P}^U$ )
17   parallel foreach  $col$  in  $Cols$  do
18     macros = getMacrosIn( $col$ );
19     sortByY( $macro$ );
20     macro2SiteExactMap = DPBasedLegalization( $macro$ ,
         $col$ );
21    $\lfloor E_p^{Legal} = linkToExactLoc(macro2SiteExactMap)$ 

```

图 5: 论文 Algorithm 2: Progressive Macro Legalization, 直接裁取自原论文。

这里最重要的不是记伪代码, 而是理解 “为什么 progressive”:

1. **Rough legalization:** 释放 macro 内 standard cells 的 shape constraints, 为每个 cell 找 candidate sites, 构造 bipartite graph, 用 min-cost bipartite matching 做 cell-to-site 映射;
2. 如果平均 displacement 还大于 θ_{exact} , 只把 matching 后 sites 的平均位置作为 anchor, 通过 pseudo net 继续平滑牵引;
3. 当平均 displacement 足够小, 再进入 **Exact legalization**, 要求 macro cells 位于同一资源列的相邻 sites。

一句话总结

Rough stage 先回答 “这个 macro 大概应该靠近哪一片合法资源？”；Exact stage 才回答 “它最终占用这一列中的哪几行？”。这样避免在 placement 还不稳定时过早做强制离散决定。

8 Exact Legalization 的 DP: 把二维问题压成列内一维问题

programming (DP):

$$f(i, j) = \min(f(i-1, j - \text{row}(i)) + \text{HPWL}^+(i, j - \text{row}(i) + 1), f(i, j-1)) \quad (9)$$

where $f(i, j)$ represents the increase of HPWL to legalize the 0- i th macros in 0- j th rows of the column, $\text{row}(i)$ represents the number of adjacent rows which should be occupied by macro i , and HPWL^+ denotes the HPWL increase when macro i is placed from row $j - \text{row}(i) + 1$ to row j , compared to the placement before legalization. Variable i will iterate from 0 to $N_{\text{macro}, \text{col}} - 1$ and Variable j will iterate from 0 to $N_{\text{site}, \text{col}} - 1$. Constraints and initial state settings for Eqn 9 are not shown because of limited space. Since the macros are close to their potential legal positions and the placement tends to be stable at this stage, we assume that the locations of the other instances will remain unchanged during the intra-column legalization of macros. Therefore, the intra-column legalization procedures for different columns can be parallelized (line 17-21 in Algorithm 2).

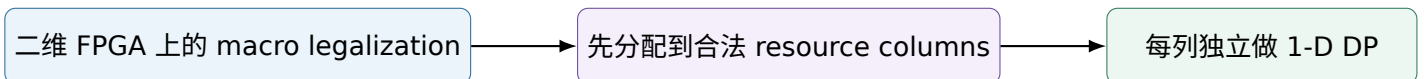
图 6: 论文 Eq. (9) 及其解释, 直接裁取自原论文。

对某一个资源 column, macros 按当前 y 坐标排序并保持相对顺序。定义:

$$f(i, j) = \min(f(i-1, j - \text{row}(i)) + \text{HPWL}^+(i, j - \text{row}(i) + 1), f(i, j-1)).$$

两个分支分别表示:

- 放置 **macro** i : 让它占据截至 row j 的连续 $\text{row}(i)$ 行;
- 暂时不用 **row** j : 继续在前 $j-1$ 行里寻找最优解。



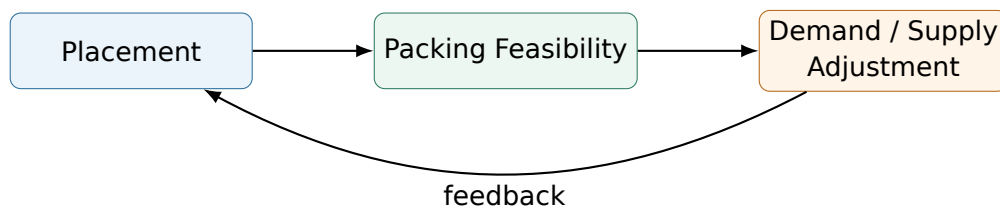
不同 columns 的 legalization 可以并行, 因为论文此时假定其它实例位置保持不变。

9 Packing 与 Resource Feedback: 为什么它们不是收尾杂项?

Final Packing 的任务是把 instances 真正映射进具有固定资源数量和类型的 FPGA sites。论文基于 extended UTPlaceF 的并行 packing, 并针对 mixed-size 做三项修改:

1. macro legalization 已经 pre-packed 的 CLB, 只要规则允许, 仍可继续填入其它 instances;
2. 对 candidate packing cluster 做 hash encoding, 减少重复 packing 尝试;
3. 对 exception instances 逐步增大 ripping-up window, 并并行处理相互独立的异常实例。

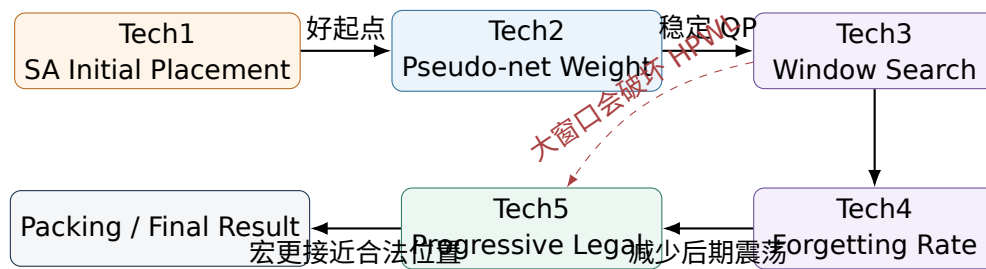
Resource Demand/Supply Adjustment 则把 packing feasibility 与 routing congestion 反馈给 placement, 使 LUT/FF 等的 demand 和区域 supply 可以调整。



论文边界

AMF-Placer 本文没有重新展开 incremental packing 的 pairing score, 也没有给出 resource demand/supply adjustment 的完整数值公式。论文明确说明这些机制分别来自 RippleFPGA 与 extended UTPlaceF, 并做了 mixed-size 适配。因此读源码时, 这两部分需要结合对应引用工作, 不能只依赖本文。

10 把 Tech1-Tech5 放到同一张因果图中



论文实验中明确提到的“恶性循环”

如果没有 Tech3, spreading 可能使用更大的窗口; 如果同时没有 Tech4, 位置会被更直接地更新并进一步增加 wirelength; 随后没有 Tech5 的 direct macro legalization 又会造成更大的 macro displacement。最终坏的 global placement 还会让 parallel packing 更难、runtime 更长。

这说明 Tech1-Tech5 并不是五个彼此独立的“小技巧”, 而是在同一个 lower-bound / upper-bound 收敛闭环里互相放大或缓解影响。

11 实验复盘：Table II / III 应该怎样读？

TABLE II: COMPARISON OF RUNTIME AND HPWL WITH VARIOUS BENCHMARKS AND CONFIGURATION

	Rosetta FaceDetect [33]				SpooNN [29]				OptimSoc [31]				MiniMap2 [15]			
	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}
proposed	446908	1.000	105	1.054	339764	1.000	129	1.000	1771538	1.000	466	1.000	1275346	1.000	558	1.000
w/o Tech1	479221	1.072	109	1.088	360567	1.061	132	1.023	9034564	5.100	871	1.870	5132073	4.024	1265	2.266
w/o Tech2	465208	1.041	131	1.315	491385	1.446	517	4.007	1841786	1.040	577	1.239	1790566	1.404	1019	1.826
w/o Tech3	487060	1.090	124	1.238	431386	1.270	240	1.857	1825819	1.031	543	1.164	1307249	1.025	626	1.121
w/o Tech4	450199	1.007	100	1.000	351649	1.035	131	1.015	2658863	1.501	525	1.126	1291565	1.013	561	1.005
w/o Tech5	511514	1.145	134	1.344	443765	1.306	138	1.067	1880431	1.061	546	1.172	1430330	1.122	708	1.268
baseline	794201	1.777	234	2.338	1863746	5.491	339	2.549	2231978	1.260	559	1.199	11886747	9.320	3539	6.339
	OpenPiton [32]				Rosetta DigitRecog [33]				MemN2N [29]				BLSTM [30]			
proposed	1139189	1.000	283	1.035	726929	1.000	254	1.097	801403	1.000	318	1.130	484650	1.000	326	1.241
w/o Tech1	6218009	5.458	350	1.280	768796	1.058	256	1.105	963416	1.202	363	1.287	492194	1.016	285	1.083
w/o Tech2	1159003	1.017	326	1.190	730071	1.004	321	1.389	916176	1.143	411	1.460	496540	1.025	364	1.383
w/o Tech3	1157479	1.016	296	1.079	773957	1.065	272	1.174	862212	1.076	343	1.216	511861	1.056	296	1.126
w/o Tech4	1212149	1.064	274	1.000	728094	1.002	231	1.000	822742	1.027	282	1.000	653010	1.347	307	1.169
w/o Tech5	1142927	1.003	301	1.098	997207	1.372	244	1.053	923670	1.153	353	1.252	722495	1.491	263	1.000
baseline	1432535	1.258	297	1.086	1047885	1.442	298	1.288	1610425	2.010	547	1.942	907383	1.872	325	1.237

TABLE III: ACCELERATION RATIOS WITH PARALLELISM

	Rosetta FaceDetect	SpooNN	OptimSoc	MiniMap2	OpenPiton	Rosetta DigitRecog	MemN2N	BLSTM
8 threads	2.17x	2.07x	2.50x	2.86x	2.63x	2.22x	2.61x	2.23x

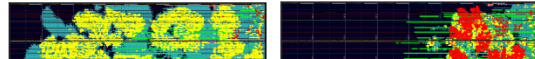


图 7: 论文 Table II 与 Table III: 消融结果和多线程加速, 直接截取自原论文。

论文报告, 与其构造的 baseline 相比, 完整方法的 HPWL 改善范围为 20.4%–89.3%, runtime 降低 8.0%–84.2%。多线程方面, 平均加速比为:

$$2 \text{ threads} : 1.64\times, \quad 4 \text{ threads} : 2.20\times, \quad 8 \text{ threads} : 2.41\times.$$

为什么不是 8 线程接近 $8\times$?

论文的解释是: global placement iterations 占主要 runtime, 而随着 overflow regions 被逐步处理, 能够形成的 non-overlapping spreading windows 很快减少。后期并行任务数量不足, 因此线程数继续增加时边际收益下降。

12 Fig. 6 与论文边界：AMF-Placer 并不等于 Vivado placer

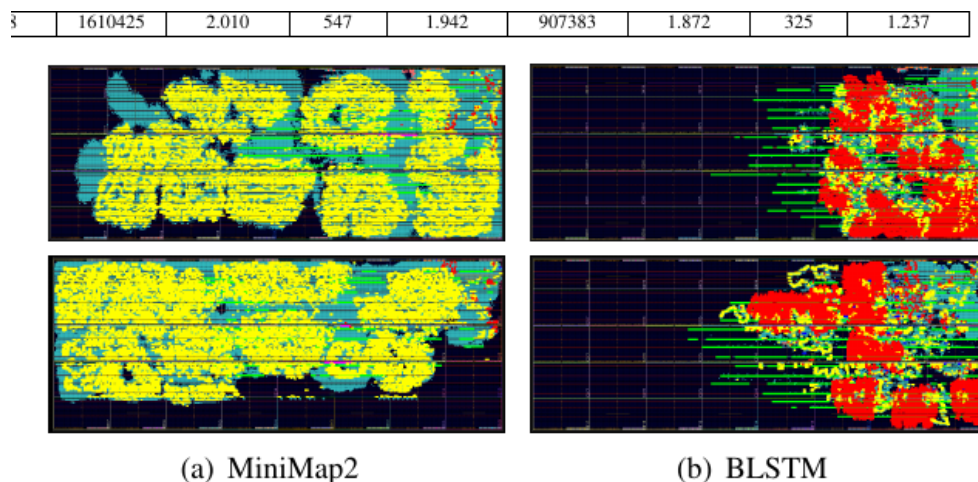


Fig. 6. Comparison of AMF-Placer Placement (upper ones) and Vivado Placement (lower ones): yellow for CARRY macros, red for MUX macros, green for BRAM macros, purple for DSP macros, blue for LUTRAM macros. The view of device is rotated left by 90°.

contests. The placement of the largest benchmark, MiniMap2 [15], and the benchmark with the highest macro ratio, BLSTM [30], are shown in Fig. 6 as examples. Different types of macros are highlighted in different colors. Due to limited space, more detailed statistics are available in the open-source

图 8: 论文 Fig. 6: AMF-Placer 与 Vivado placement 的可视化对比, 直接裁取自原论文。

论文指出, AMF-Placer 生成的 placement 可以加载到 Vivado 并成功完成 routing; 其与 Vivado 的 congestion extent 差异较小。但作者同时明确承认:

- Vivado 使用更好的 floorplanning, 并利用 design hierarchy, 因此 AMF-Placer 的 slice utilization 略高;
- Vivado placer 是 timing-driven, 而 AMF-Placer 当前是 wirelength-driven;
- 因而 AMF-Placer 的 WNS/TNS 结果比 Vivado placement 差。

必须记住的研究边界

这篇论文证明的是: AMF-Placer 能高效处理现代 FPGA 上的大规模 mixed-size analytical placement, 并在其 baseline 上显著改善 HPWL/runtime。它并没有证明自己已经解决完整的 timing closure, 也没有声称在 timing 上优于 Vivado。

13 哪些部分是本文重点创新，哪些是继承已有工作？

模块	论文中的来源/定位	Day 6 应怎样记住
SA-based Initial Placement	本文强调的 mixed-size 初始布局策略；cluster partition 使用 PaToH，并采用 clock-aware criteria	Tech1：用大范围随机搜索避开异构资源导致的坏局部区域
Quadratic Placement	B2B quadratic model 延续 analytical placer 思路；本文加入 legalization pseudo nets 与 interconnection-density-aware weight	Tech2：高连接度 macro 更强地被 anchor 稳定
Cell Spreading	基础 bi-partitioning spreading 来自已有工作；本文加入 mixed-size deadlock、window search、fluctuation、forgetting-rate 等处理	Tech3/4：控制 spreading 的方向、范围和步长
Progressive Macro Legalization	本文核心 mixed-size 机制	Tech5：rough matching → anchor → exact column DP
Incremental Packing	pairing algorithm 来自 RippleFPGA	论文只交代功能，不应自行补 pair-score 细节
Resource Demand/Supply + Final Packing	主要来自 extended UTPlaceF，并针对 mixed-size 修改	是实现性反馈和最终离散化的重要组成部分，但本文不是其完整算法出处

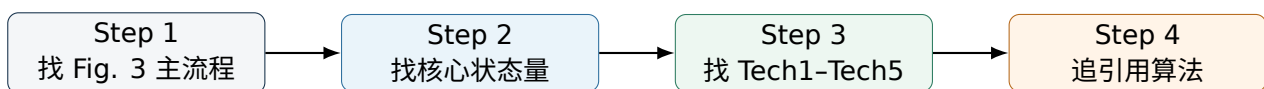
为什么这一页对读源码很重要？

如果不知道算法来源，很容易在代码里把“继承的基础设施”当成本文贡献，或者反过来遗漏 AMF-Placer 真正针对 mixed-size 新增的逻辑。进入源码前应先把这张表记住。

14 Paper-to-Code: 下一阶段怎样开始读源码?

本论文只给出了开源仓库地址，并没有在正文中描述目录结构。因此下面不是对仓库文件名的断言，而是一套基于论文术语的代码检索顺序。

1. 先找总流程入口。搜索 “initial placement”、“quadratic placement”、“cell spreading”、“macro legalization”、“packing” 等阶段名，确认 Fig. 3 在代码里怎样串起来。
2. 再找关键状态量。搜索 $HPWL_{lower}$ 、 $HPWL_{upper}$ 、movement、pin number、overflow bin、candidate site、pseudo net、anchor、forgetting rate 等术语或对应变量。
3. 优先定位 **Tech1-Tech5** 的差异点。不要一开始深入所有基础数据结构；先找到论文新增机制在哪里插入已有 placer 框架。
4. 最后追外部来源。对 RippleFPGA pairing、extended UTPlaceF demand/supply/final packing 等本文未展开部分，再结合引用论文阅读实现。



15 源码阅读时最值得打断点/打印日志的量

这一页是基于论文算法关系给出的**调试观察清单**，不是论文声明的具体代码接口。它的目的，是让源码阅读有明确问题，而不是从类定义开始漫游。

阶段	建议观察的量	想验证的问题
Initial Placement	cluster size、bin assignment、SA cost、overflow term	初始解是怎样兼顾 wirelength 和资源拥挤的？
Quadratic Placement	movement、pseudo weight、pin-Num、anchor locations	高 pin-count macro 是否真的移动更保守？
Cell Spreading	overflow times、window bounds、neighbor utilization、 γ	deadlock、窗口扩张和后期保守更新是怎样发生的？
Macro Legalization	θ_{disp} 、candidate sites、matching cost、avgDisp、 θ_{exact}	什么时候从 rough 切换到 exact？
Exact DP	macro order、row(i)、 $HPWL^+$ 、DP table	列内 DP 如何保持顺序并选择连续 site 区间？
Packing	candidate cluster、hash、exception count、rip-up window	hash 去重和异常修复到底节省了什么？

建议的第一个源码实验

选一个较小 benchmark，只运行到 global placement；每轮记录 $HPWL_{lower}$ 、 $HPWL_{upper}$ 、overflow cell 数量、宏 rough/exact legalization 数量。这样可以直接把代码运行轨迹映射回 Fig. 3，而不是一开始就追全部 C++ 细节。

16 如果要复现实验，最小闭环应该是什么？

完全复现论文所有 benchmark 与 Vivado flow 的成本较高。仅从论文结构出发，一个最小的学习型复现可以按下面顺序组织：

1. 确认一个 design 可以从 Vivado pre-implementation netlist 进入 AMF-Placer, 并导出 placement Tcl;
2. 跑完整 proposed configuration, 记录 HPWL 与 runtime;
3. 按论文 Tech1-Tech5 逐项关闭, 至少做一个小型 ablation;
4. 对比 proposed / w/o Tech2 / w/o Tech3 / w/o Tech4 / w/o Tech5 的 macro displacement 与 overflow evolution;
5. 最后把 placement 加载回 Vivado, 确认能 route; 不要把 HPWL 改善自动等同于 timing 改善。

为什么先做“轨迹复现”，再做“最终数字复现”？

对学习源码而言，首先验证算法机制是否按论文描述发生，比一开始追求完全一致的 Table II 数字更重要。例如先看到 pseudo weight 随 pinNum 增强、 γ 随收敛变小、avgDisp 触发 rough-to-exact 切换，这些现象能直接帮助理解实现。

17 Day 6 自测：你现在是否真正掌握了整篇论文？

请尽量不翻前面内容，回答下面问题：

1. 为什么 modern FPGA 的 macro 不能简单当成 “面积更大的 standard cell”？
2. 为什么 AMF-Placer 在 QP 前先做 cluster-level SA？
3. pseudo net 同时承担哪两类作用？为什么新的权重中要乘 $pinNum$ ？
4. macro 为什么会造成 cell spreading deadlock？resource supply fluctuation 想打破什么现象？
5. utilization-guided window search 与 forgetting rate 分别控制 spreading 的什么方面？
6. progressive macro legalization 为什么不能一开始就 exact legalize？
7. rough legalization 中 bipartite matching 的 edge cost 是什么？
8. Eq. (9) 的两个 DP 分支分别代表什么？为什么不同 columns 可以并行？
9. Incremental Packing、Final Packing、Resource Demand/Supply Adjustment 中，哪些细节本文并没有完整展开？
10. 为什么论文的 HPWL 改善不能直接推出 WNS/TNS 一定更好？

如果上面 10 题能独立回答

你已经不只是 “读过 AMF-Placer”，而是建立了可以进入源码的算法模型。下一阶段再读代码时，重点将从 “这段代码在做什么” 转向 “它对应论文里的哪一个状态、哪一个近似、哪一个收敛机制”。

18 六天学习总览与下一步

学习日	核心内容
Day 1	Mixed-size placement 问题、macro 类型、7 阶段整体框架
Day 2	SA-based Initial Placement、B2B Quadratic Placement、pseudo nets、 <i>pinNum</i> 权重
Day 3	Cell Spreading、macro deadlock、resource fluctuation、window search、forgetting rate
Day 4	Progressive Macro Legalization、bipartite matching、rough-to-exact、column DP
Day 5	Demand/Supply Adjustment、Incremental/Final Packing、实验结果与 Vivado 边界
Day 6	全景复盘、Tech1-Tech5 耦合关系、paper-to-code 阅读准备

下一阶段建议：进入 AMF-Placer 源码学习。

第一天源码阅读不建议直接钻进某个 solver。更有效的路线是：先找到 Fig. 3 的顶层迭代入口，画出调用图；然后只跟踪一个 benchmark 在 Initial Placement → QP → Spreading → Macro Legalization 之间的数据流。

来源边界说明：本文所有关于 AMF-Placer 算法、实验数字和论文结论的陈述均基于所提供论文。“Paper-to-Code”与调试观察项是为了下一步阅读实现而整理的学习方法，并非论文对具体仓库目录、函数名或变量名的声明。